

how maneha recap vidoe get made?

Bro, Manhwa recap videos are usually made by taking a manhwa/manhua story, converting it into a narration script, adding visuals, AI voiceover, and editing everything into a YouTube video.

Since you're creating Manhwa/Manhua recap videos for your channel, let me explain the complete process in simple steps.

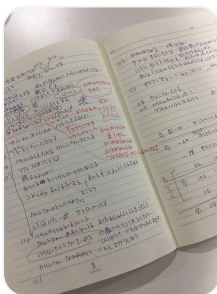
How Manhwa Recap Videos Are Made



1. Find the Manhwa / Manhua Story [Story Source](#)

Creators choose a manhwa, manhua, webtoon, or related story with interesting characters, conflict, and plot twists.

Example: A weak prince gets a system and becomes the strongest emperor.



2. Read the Story and Create a Recap Script

The creator understands the plot and rewrites it into an engaging narration.

Example:

"Everyone believed the prince was useless. But nobody knew he had awakened a mysterious system."

The script should explain the story clearly rather than simply copy dialogue from the comic.

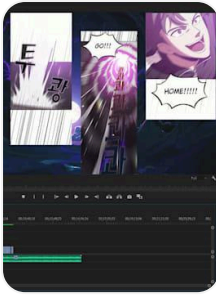


3. Generate the Voiceover

The narration is converted into spoken audio using a human narrator or an AI voice tool.

Common AI voiceover options include ElevenLabs and other text-to-speech services.

Good pacing, pronunciation, and emotional delivery help make the story easier to follow.

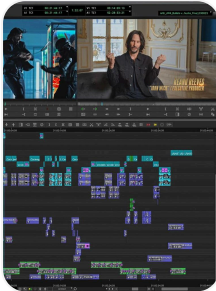


4. Collect and Prepare Visuals

Creators prepare comic panels, illustrations, backgrounds, and other visuals to accompany the narration.

Common editing techniques include:

- Zooming into panels
- Panning across artwork
- Cropping and reframing
- Adding transitions and visual effects



5. Edit the Video

The creator combines voiceover, visuals, music, sound effects, and subtitles.

Popular editing tools include:

- Adobe Premiere Pro
- CapCut
- DaVinci Resolve



6. Create the Thumbnail and Upload

The creator prepares a title, thumbnail, description, and other upload details.

Example:

He Was the Weakest Prince... Until His System Awakened!

Complete Manhwa Recap Production Workflow

From Story to YouTube Video

A typical AI-assisted recap workflow

1. Story / Source

Manhwa, manhwa, or source video



2. Story Understanding

Read, transcribe, or analyze the story



3. AI Script Writing

Narration, hooks, and scene breakdown



4. AI Voiceover

Generate natural narration



5. Video Editing

Visuals + audio + subtitles + effects



6. Thumbnail & SEO

Title, thumbnail, description, upload

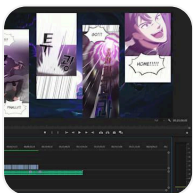


7. Publish on YouTube

Review audience retention and improve future videos

Different Types of Manhwa Recap Creators

There are several production approaches. They don't all make videos in exactly the same way.



1. Traditional Recap Creator

Uses a human-written script, recorded narration, and manually edited comic panels.

More hands-on production.



2. AI-Assisted Recap Creator

Uses AI for script drafts, voiceover, image preparation, subtitles, and parts of the editing workflow.

Human review is still important for accuracy and quality.



3. Source-Video Adaptation

Starts from an existing drama or recap video, then may translate, rewrite, dub, and re-edit it.

Requires attention to copyright and meaningful originality.

For Your ManhwaRecapGod Channel

Based on your goal of creating English recap videos using AI and existing source material, your workflow could look like this:

Step	Your production
Story	Existing manhwa / Chinese AI drama source
Script	AI-generated English recap
Voice	AI English narration
Visuals	Source video or prepared visuals
Editing	Improve pacing, subtitles, effects
Thumbnail	AI-assisted custom thumbnail
SEO	Title, description, and relevant tags

Important: If you're adapting existing videos, simply changing the voiceover or adding minor edits may not provide enough originality or rights to reuse the content. Copyright permissions and YouTube's monetization policies should be considered.

What do you want to learn next?

Choose what you want to know

- ☐ How professional Manhwa recap channels make videos
- ☐ How to make Manhwa recap videos using AI from start to finish
- ☐ How creators convert existing Chinese AI drama videos into English recaps
- ☐ How to automate the entire process using Python + AI
- ☐ How YouTube recap channels create videos like Duskpage



Your Free AI-Assisted Manhwa Recap Production System

Bro, let's design a practical **Manhwa Recap Generator** using **Python + AI + comic panels**, focused on producing **5–10 minute English recap videos** without requiring paid AI subscriptions.

Your goal is to automate repetitive work while keeping control over the story, narration quality, visual timing, and final video.

Free-first Python AI-assisted 5–10 min videos

Scope: Manhwa/manhua panels → English script → scene matching → video assembly → reviewable MP4.

1. Recommended Architecture

Manhwa Recap Pipeline

Local-first

📁 Input: Manhwa Panels

Images, chapter folders, optional source transcript



OCR + Text Extraction

Extract dialogue and panel metadata



Story Understanding + Translation

Translate, organize events, track characters



Recap Script Generator

Natural English narration with scene references



Narration / Timing

Optional local TTS or recorded narration; estimate duration



Visual Scene Matching

Map narration segments to relevant panels



Python Video Assembly

Pan/zoom, transitions, subtitles, audio



Output + Human Review

MP4, subtitles, scene manifest, and quality checks

Design principle: Build the pipeline in independent modules. You should be able to regenerate the script without reprocessing all images, or change the video timing without rewriting the story.

2. Free Tool Stack

The following setup prioritizes free and open-source tools that can run on your Windows computer. "Free" means no mandatory paid API subscription for the core workflow; local

AI models may require sufficient RAM, storage, and potentially a GPU.

Recommended tools



Python Core

Purpose: Pipeline orchestration, scene metadata, automation, and file management.

Use Python 3.11 or a compatible version supported by your chosen dependencies.



PaddleOCR

Purpose: OCR for dialogue and text extraction.

A multilingual OCR toolkit. Check its current installation requirements for your Windows CPU/GPU setup. [PaddleOCR Documentation](#) +1



Ollama

Purpose: Run a locally installed language model for story analysis and script drafting.

Requires a suitable local model and enough system resources. Model selection depends on your computer.



FFmpeg

Purpose: Video encoding, audio merging, resizing, and media processing.

Use it as the underlying rendering tool from Python.



OpenCV

Purpose: Image processing, resizing, cropping, and optional visual analysis.

Useful for panel preparation and custom rendering operations.



MoviePy

Purpose: Python-based video composition.

Can help assemble clips, images, and audio. You can also directly invoke FFmpeg for more predictable rendering.

Optional: Existing open-source project to study

ACMP is an open-source project designed to convert static manga/manhwa/comic images into motion-comic videos. Its documented architecture includes panel processing, scene

analysis, Ken Burns animation, and FFmpeg encoding. It could serve as a reference for your rendering module rather than something you must build from scratch.  [GitHub](#)

3. Project Folder Structure

Start with a modular Python project.

```
manhwa_recap_ai/
├── app.py
├── requirements.txt
├── .env.example
├── input/
│   ├── project_001/
│   │   ├── panels/
│   │   │   ├── page_001.jpg
│   │   │   ├── page_002.jpg
│   │   │   └── page_003.jpg
│   │   └── source_text.txt
├── data/
│   ├── project_001/
│   │   ├── ocr.json
│   │   ├── story.json
│   │   ├── script.json
│   │   └── scene_map.json
├── src/
│   ├── ingest.py
│   ├── ocr.py
│   ├── story_analyzer.py
│   ├── script_writer.py
│   ├── scene_matcher.py
│   ├── timeline.py
│   ├── renderer.py
│   └── validators.py
├── assets/
│   ├── fonts/
│   ├── music/
│   └── transitions/
├── output/
│   ├── project_001/
│   │   ├── preview.mp4
│   │   └── final.mp4
```



```
|  
└─ logs/
```

Why this structure?

You can reuse the same OCR output and story analysis if you decide to regenerate only the narration. This avoids repeating every processing step for small changes.

4. Phase 1 — Import and Organize Panels

Input

You provide a folder of comic panels.

Example:

```
input/project_001/panels/  
  page_001.jpg  
  page_002.jpg  
  page_003.jpg  
  page_004.jpg
```



Processing tasks

1. Scan the folder.
2. Sort the files by filename.
3. Validate supported image formats.
4. Read image dimensions.
5. Create unique panel IDs.
6. Store metadata in JSON.

Example metadata

```
</> JSON  
  
{  
  "panel_id": "page_001",  
  "file": "page_001.jpg",  
  "order": 1,  
  "width": 1920,  
  "height": 1080  
}
```



Implementation tip: Preserve the original files and create a separate processed-images folder. Avoid overwriting the source artwork.

5. Phase 2 — OCR and Story Data

You selected transcription and translation, but your source is **comic panels**, so the first task is OCR rather than audio transcription.

Pipeline

Comic Panel

↓ Text detection and recognition

OCR

↓ Review recognized text

Translation

↓ Associate text with panel ID

Structured Story Data

Example OCR output

 JSON 

```
{
  "panel_id": "page_001",
  "language": "zh",
  "text": [
    {
      "original": "你终于回来了",
      "translation": "You're finally back.",
      "confidence": 0.92
    }
  ]
}
```

The confidence value here is an illustrative field; actual OCR confidence should come from your OCR implementation and must not be assumed to be accurate without validation.

Tool choice

For Chinese or multilingual comic text, evaluate PaddleOCR's language support and actual recognition quality on your panels. OCR output should be manually reviewed, especially for stylized fonts, overlapping speech bubbles, and small text.

Practical MVP recommendation: Allow manual text correction through a JSON file or a simple review screen before the script generator uses the extracted text.

6. Phase 3 — Story Analysis

This module converts panel-level information into a coherent story summary.

Input

- OCR results
- Panel ordering
- Any user-provided plot notes
- Optional original synopsis

Output

A structured story document.

</> JSON



```
{
  "title": "Unknown",
  "characters": [
    {
      "id": "char_001",
      "name": "Main Character",
      "role": "Protagonist"
    }
  ],
  "events": [
    {
      "event_id": "event_001",
      "panel_ids": ["page_001", "page_002"],
      "summary": "The protagonist returns to the village.",
      "importance": "high"
    }
  ],
  "conflicts": [],
  "unresolved_questions": []
}
```

AI prompt for story analysis

You are a story analyst for a Manhwa recap channel.



Analyze the provided panel OCR and metadata.

Goals:

1. Identify the main characters.
2. Identify the order of major events.
3. Separate confirmed events from uncertain OCR.
4. Track character relationships and names.
5. Identify important conflicts and reveals.
6. Do not invent scenes, dialogue, powers, or events.

Return valid JSON using this schema:

- characters
- events
- conflicts
- unresolved_questions

Each event must include the relevant panel_ids.

Critical rule: OCR text alone may not tell you what is happening visually. The model should not treat incomplete dialogue as the complete story. Use panel images or human notes to verify important plot details.

7. Phase 4 — Generate a Human-Style Recap Script

This is the part where your channel's narration quality can improve.

You should not make the model simply translate every speech bubble. Instead, use the story events to create a flowing recap that preserves the actual plot.

Script format

For a 5–10 minute video, aim for a script length appropriate to your chosen narration speed. For example, at 150 words per minute:

Duration	Approx. words
5 minutes	750
7 minutes	1,050
10 minutes	1,500

These are planning estimates, not mandatory word counts. Pauses, dialogue, and delivery speed will change the actual duration.

Example output schema

</> JSON



```
{
  "title": "He Was the Weakest Prince... Until His System Awakened!",
  "target_duration_sec": 420,
  "segments": [
    {
      "segment_id": "seg_001",
      "event_ids": ["event_001"],
      "panel_ids": ["page_001", "page_002"],
      "narration": "After years of being forgotten, the prince finally returned..",
      "estimated_duration_sec": 6.5
    }
  ]
}
```

Script-writing prompt

You are an English Manhwa recap scriptwriter.



Write a natural, engaging recap narration from the verified story events.

Requirements:

- Use clear conversational English.
- Start with an engaging hook based on confirmed story details.
- Keep the story in chronological order unless a flashback is clearly ident
- Avoid inventing events or character motivations.
- Do not repeat every speech bubble.
- Preserve important names, relationships, and plot details.
- Reference relevant panel_ids for every segment.
- Keep the tone suitable for a YouTube recap.
- Avoid excessive filler and repetitive expressions.

Output valid JSON with:

- title
- target_duration_sec
- segments
 - segment_id
 - event_ids
 - panel_ids
 - narration
 - estimated_duration_sec

Example

Generic translation:

The prince goes to the palace. He sees the king.

Recap narration:

After years of being treated like a forgotten nobody, the prince finally steps inside the royal palace. But this isn't the homecoming he imagined. The king has summoned him for a reason—and it could change his entire fate.

The second version is an illustrative writing style, not a claim about a specific story. Your system should generate such narration only from the verified plot.

8. Phase 5 — Visual Scene Matching

This is one of the most important modules for your project.

Your script contains narration, but your video needs to know **which panel should appear during each sentence**.

Instead of relying entirely on an AI vision model, use a structured approach.

Scene matching architecture

📄 Narration Segment

"The prince finally enters the royal palace."



🔍 Candidate Panel Search

Find panels whose story events and metadata match the narration.



✅ Human / AI Validation

Confirm that the selected panel actually depicts the described scene.



📅 Timeline Assignment

Assign the panel, duration, and motion effect to the video segment.

Three matching methods

Method 1: Panel IDs from the Script

MVP

The script generator directly references the panel IDs that support each event.

Simple, inexpensive, and easy to debug. Requires accurate story analysis.

Method 2: Keyword / Event Matching

Phase 2

Match event tags such as `palace`, `battle`, `awakening`, or `meeting` against story metadata.

Useful as a fallback when a script segment has no explicit panel IDs.

Method 3: AI Vision Matching

Advanced

Use an image-capable local model to analyze candidate panels and compare them with a narration segment.

More complex and computationally demanding. Keep human verification for important scenes.

My recommendation: Start with Method 1. It is more predictable than trying to automatically match every narration sentence to every image from day one.

9. Phase 6 — Video Rendering With Python

You want to convert static comic panels into a video with motion.

For a free-first workflow, begin with:

- Static panel images
- Ken Burns-style zoom and pan
- Crossfades or simple transitions
- Narration audio
- Optional subtitles
- Background music only when you have appropriate usage rights

Timeline example

Example: 30-Second Scene

Illustrative

Narration

0–30 sec

English voiceover

Visuals

0–30 sec

P1

P2

P3

P4

Panel IDs and durations are illustrative; actual durations come from your narration and timeline decisions.

Recommended rendering sequence

1. Load a panel image.
2. Resize it to the target video dimensions.
3. Apply a slow zoom or pan.
4. Crop to the target aspect ratio.
5. Render the image clip for the assigned duration.
6. Combine all clips.
7. Add voiceover and subtitles.
8. Export through FFmpeg.
9. Validate the output.

Example Python dependencies

```
Pillow  
opencv-python  
moviepy  
pydantic
```



You can add these to your `requirements.txt`, then install them inside a virtual environment.

</> Bash

```
python -m venv .venv  
.venv\Scripts\activate
```

```
pip install pillow opencv-python moviepy pydantic
```



Note: MoviePy's API varies by version. Pin and test the version you choose instead of blindly copying examples written for older releases.

10. Video Editing Strategy

For the first version, do not try to build a complete AI video editor. Build a reliable motion-comic renderer.

Version 1 features

- ✔ Load panels from a folder.
- ✔ Read a structured scene JSON file.
- ✔ Render a zoom/pan effect.
- ✔ Merge narration audio.
- ✔ Export MP4.
- ✔ Generate a preview for review.

Version 2 features

- Automatic subtitle generation from narration.
- Better panel selection using event metadata.
- Scene transition presets.
- Background music track management.
- Video preview interface.
- Batch rendering for multiple episodes.

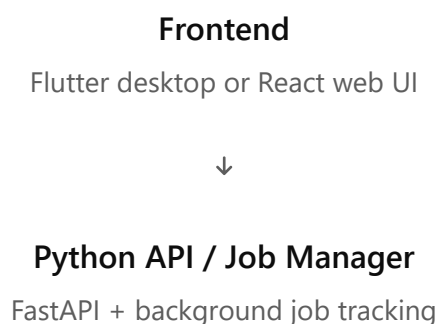
Version 3 features

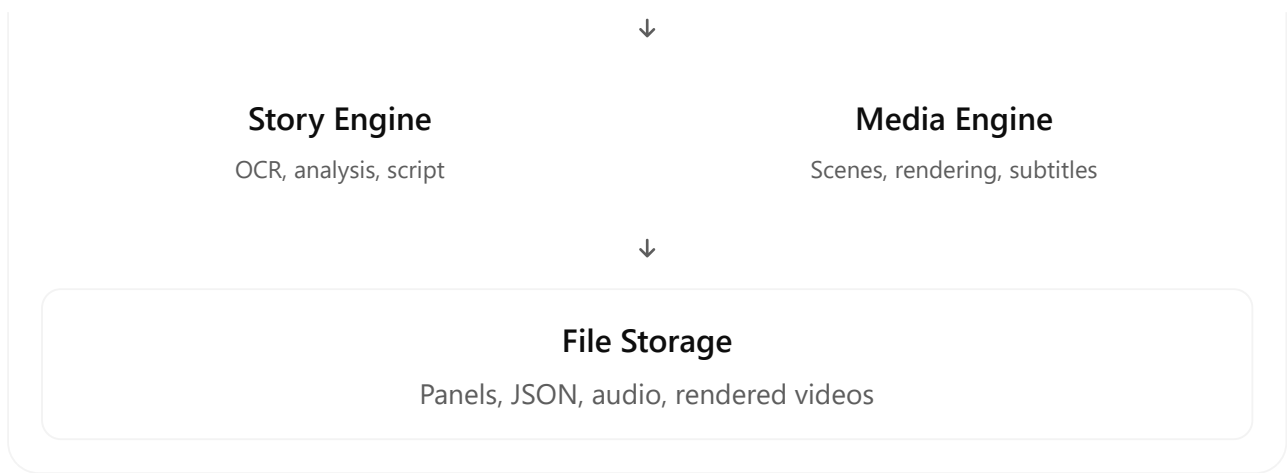
- AI-assisted scene matching.
- Character tracking and consistency checks.
- Automatic pacing suggestions.
- Story continuity checks.
- Review dashboard for rejected or uncertain scenes.

11. Project Architecture for a Future UI

Since you have Flutter and backend experience, you could later create a local web or desktop dashboard on top of the Python engine.

Optional Application Architecture





You don't need FastAPI or a frontend for the first prototype. A command-line Python application will be easier to test. Add the UI once the core pipeline produces a good video.

12. Implementation Plan

Here is the order I recommend for building it.

1 Input + Panel Manager

First

Build the folder scanner, image metadata, sorting, and validation.

Deliverable: Validated panel dataset.

2 Story Data + Script

Build the JSON schema and manually test it with a short story.

Deliverable: Structured recap script with panel references.

3 Rendering Engine

Create a motion-comic video from a manually written scene JSON file.

Deliverable: A short MP4 using test images.

4 Audio + Subtitle Integration

Add narration audio, duration adjustment, and caption support.

Deliverable: Narrated scene with synchronized subtitles.

5 OCR + AI Assistance

Integrate OCR, story analysis, and script generation after the renderer works.

Deliverable: AI-assisted input-to-script workflow.

6 Scene Matching + Quality Review

Add panel references, confidence flags, and a review process for uncertain scenes.

Deliverable: Repeatable production pipeline with human verification.

13. Important Free-Tier Limitations

Before you commit to a fully local AI pipeline, consider the following:

Component	Free-first consideration
OCR	Local OCR is possible, but model quality varies by language and image quality.
Script writing	Local language models require sufficient RAM and may be less capable than paid hosted models.
Translation	Local models can translate, but accuracy needs testing, especially for Chinese names and idioms.
Voiceover	Fully local TTS is possible, but naturalness and available voices vary.
Video rendering	CPU rendering can be slow for long videos or many high-resolution panels.
Storage	Source panels, model files, audio, and rendered videos can use significant disk space.

For a zero-subscription setup, prioritize local processing and design your application so an external AI provider can be added later as an optional adapter. Do not make the entire system dependent on a paid API.

14. What I Would Build First for You

Start with a working motion-comic renderer before implementing the AI story engine.

The first milestone should be:

Give Python a folder of 20–50 comic panels and a scene JSON file, and it generates a 30–60 second narrated motion-comic video.

Once this works, you can automate the script and scene references.

That approach helps you isolate problems:

- Is the video rendering correctly?
- Are panel transitions smooth?
- Is narration timing accurate?
- Are subtitles readable?
- Are the selected panels relevant?

After those pieces work, connect OCR and AI analysis.

Let's start building it

To generate the actual Python implementation for your computer, I need three details.

1. Your Python setup

- ☐ Beginner — need complete code and setup
- ☐ Intermediate — can work with Python modules
- ☐ Advanced — comfortable building pipelines

2. Your computer hardware

- ☐ Windows 10/11
- ☐ 8 GB RAM
- ☐ 16 GB RAM
- ☐ 32 GB+ RAM
- ☐ NVIDIA GPU
- ☐ Intel integrated graphics
- ☐ AMD GPU

3. Voiceover preference

- ☐ Free local AI voice (no API cost)
- ☐ I will provide my own voiceover
- ☐ Start with placeholder audio and add voice later

Build My Python Prototype →

Recommended first coding task: Build the panel-to-video renderer. It is the foundation on which the rest of your AI-assisted workflow will run.



main issue i face when i was creating something similar was clipign the manag pannel to show on vidoe , what heppen vidoe was created but image get clip at random some time half face or half builng , some tile vertical long single sceen crop into 3-5 image

Bro, **now I understand your real problem.** This is not primarily an AI script-writing issue. It is a **comic panel detection, cropping, and video framing problem.**

You already created a video, but the visuals are getting damaged:

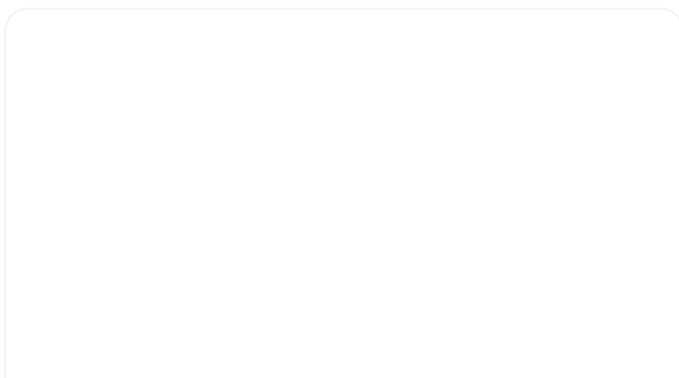
- A character's face is cut in half.
- A building is cropped randomly.
- A tall vertical comic panel gets split into 3–5 images.
- A single scene is broken into multiple crops.
- Important parts of the artwork disappear when converting to a video aspect ratio.

This happens when the system treats a complete comic page or image as something that can be freely cropped into a fixed video frame.

The solution is to separate panel detection from video framing. Your AI should understand the original image boundaries first, then decide how to display that image without destroying important content.

1. Why Your Current Cropping System Fails

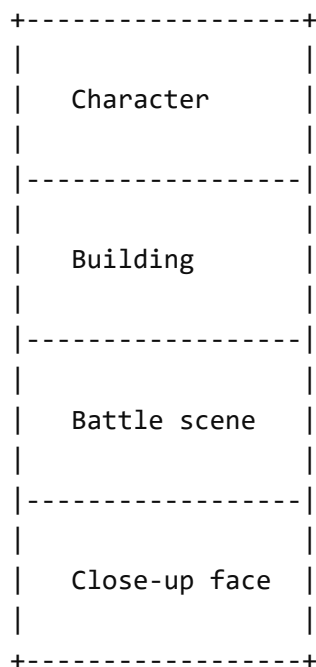
Imagine you have a vertical Manhwa page:





The original image might be:

Original Image
Width: 800 px
Height: 4000 px



If you force this image into a 16:9 video frame using a center crop, the program may effectively do this:



It doesn't understand that the character's face or building is important. It only knows the target width and height.

The 4 common causes



1. Fixed Center Cropping

A wide or vertical image is resized and cropped to fit 16:9, causing content near the edges to disappear.



2. Incorrect Panel Detection

The system treats a full page as several panels or splits a tall scene at arbitrary positions.



3. No Content-Aware Framing

The renderer does not know where the face, character, or important structure is located.



4. Tall Images Forced Into Short Clips

A long comic image is divided into arbitrary sections instead of being displayed as a complete scene or carefully selected subregions.

2. The Solution: Smart Panel-to-Video Engine

Instead of this:



Wrong Workflow

Full image



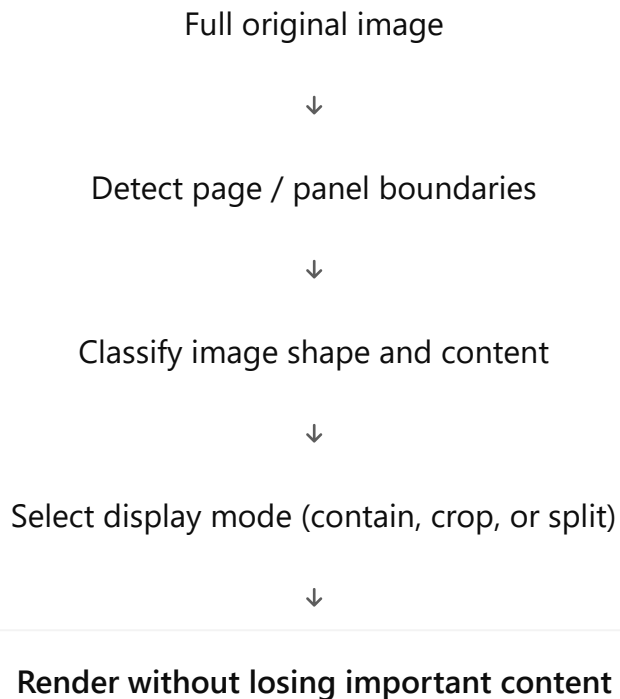
Force crop to 16:9



Face/building cut off

You need this:

✓ Correct Workflow



The key concept is:

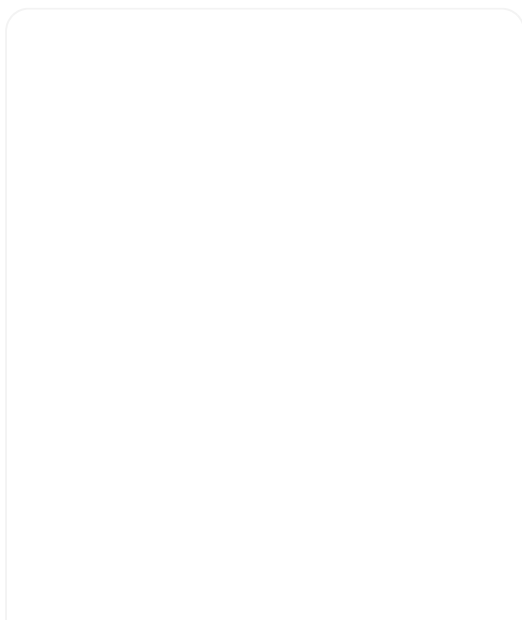
Never split or crop an image unless the system has a reason to do so.

3. Three Rendering Modes You Should Implement

Your system should not use one crop rule for every image. It should select a rendering mode based on the image's shape and whether it is a full page or an individual panel.

Mode A — Full Image Contain

Default





Use when: The image is a complete panel or a tall scene that should remain intact.

Instead of cropping, scale the complete image so it fits inside the video frame. Fill unused space with a blurred version of the same artwork or a suitable background.

- ✓ Preserves the complete image.
- ✓ Works well for tall scenes and important full-body artwork.
- ⚠ May leave empty space on the sides or top/bottom.

Mode B — Smart Crop

Optional



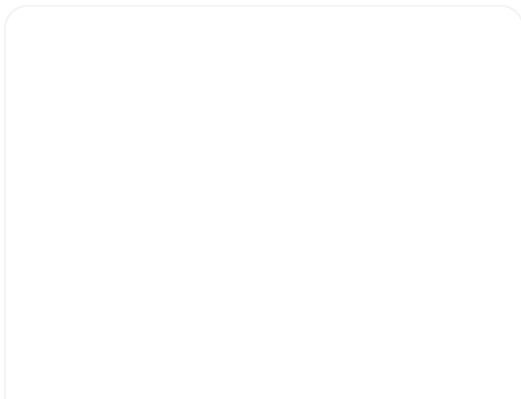
Use when: The image contains extra margins or the important subject can be safely framed inside a crop.

The crop should be calculated using a detected subject or a verified region of interest, rather than blindly cropping the center.

- ✓ Uses the available video space more effectively.
- ⚠ Requires safeguards so the face or important object is not clipped.

Mode C — Intentional Scene Split

Advanced





Use when: One tall source page contains multiple distinct comic panels or a long scene that benefits from multiple shots.

Split only using detected panel boundaries or manually confirmed regions. A tall image should not automatically be split into three to five pieces merely because it is long.

- ✓ Can create a more dynamic video from long pages.
- ⚠ Needs panel boundary detection and minimum-size checks.

My recommended priority

Mode	Priority
Full image contain	Always implement first
Smart crop	Add after safe framing works
Intentional scene split	Add after panel detection works

For your specific problem, **Mode A should be the safety fallback**. If the system is uncertain, it should preserve the full image rather than risk destroying the artwork.

4. The Most Important Fix: Aspect-Ratio-Aware Scaling

Let's say your output is 1920×1080 (16:9).

Your source image is 800×4000 (very tall).

A normal contain operation uses:

$$s = \min \left(\frac{W_v}{W_i}, \frac{H_v}{H_i} \right)$$

Where:

- W_v, H_v : Video dimensions

- W_i, H_i : Input image dimensions
- s : Scaling factor

For the example:

$$s = \min(1920/800, 1080/4000)$$

The image scales down to fit the video height, keeping its complete content visible.

The resulting image is centered with unused space around it, rather than having its top and bottom chopped off.

Important distinction

Contain preserves the full image.

Cover fills the entire video frame, which may crop some of the image.

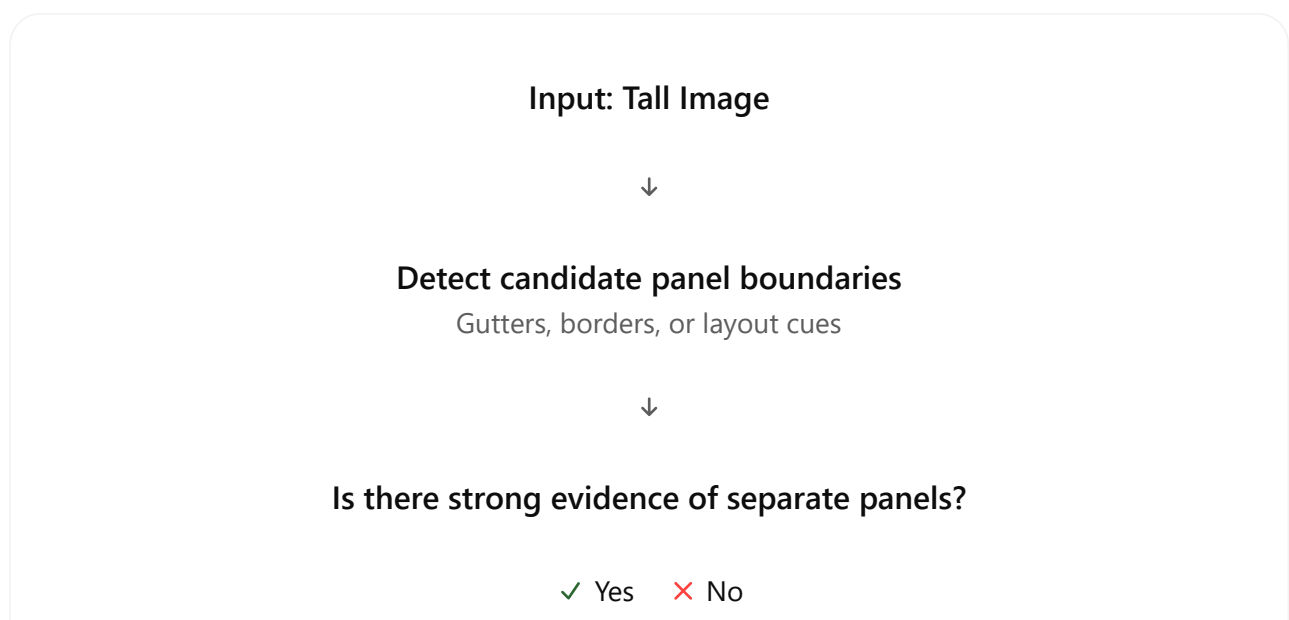
For your first implementation, use **contain** by default.

5. How to Prevent the Vertical Panel From Being Split Randomly

Your system needs a panel segmentation strategy.

A tall image can be one complete panel, or it can be a page containing several separate panels. Those are different cases.

Recommended decision process



Validate splits

Keep only meaningful boundaries

Keep as one image

Contain and preserve



Render scene safely

How to detect candidate boundaries

For a first version, you can use image processing to find possible horizontal gutters:

- Analyze horizontal lines or regions with strong separation.
- Identify gaps between image regions.
- Use minimum panel height constraints.
- Avoid treating every blank line as a valid split.
- Reject tiny or suspicious crops.

However, **simple whitespace detection is not enough for all manhwa pages**. Panels can be borderless, contain gradients, or use irregular layouts. Use it as a candidate generator, not an unquestionable splitting rule.

A stronger approach is:

1. Detect candidate boundaries.
2. Create candidate regions.
3. Validate them using a vision model or heuristic rules.
4. Keep the original page as fallback if confidence is low.

6. Prevent Half Faces and Half Buildings

This requires a **safe crop validator**.

Before cropping an image, your system should verify that important content remains inside the visible region.

Crop validation strategy

A. Face / Character Detection

If a face is detected, make sure the crop does not cut through it.

A face detector can help, but it is not guaranteed to identify every face in stylized manhwa art.

B. Subject Bounding Box

If you know the important region of the image, keep it inside the crop with a margin.

For example, preserve the full detected face bounding box plus additional padding.

C. Crop Safety Rules

Reject a crop if it cuts through a detected important region or violates minimum dimensions.

If the crop is unsafe, switch to contain mode or a wider crop.

D. Visual Review

Generate a contact sheet or preview showing the proposed crop boundaries.

This is especially useful for finding problems before rendering a complete video.

Example crop decision

Suppose the face bounding box is:

```
Face:
x = 300
y = 200
width = 250
height = 300
```



The crop must contain the face box, plus a safety margin.

If the proposed crop removes the top part of the face, the validator rejects it.

This can be implemented using OpenCV and a face detector, but a detector alone cannot reliably understand every important building, weapon, hand, or scene object. A fallback to the full image is still essential.

7. Recommended Scene Data Model

Instead of storing only a crop rectangle, store a rendering decision.

```
</> JSON

{
  "scene_id": "scene_001",
  "source_image": "page_001.jpg",
```



```

"mode": "contain",
"crop": null,
"duration_sec": 5.5,
"motion": {
  "type": "slow_zoom",
  "start_scale": 1.0,
  "end_scale": 1.08
},
"validation": {
  "status": "approved",
  "confidence": 1.0
}
}

```

For a verified crop:

</> JSON 

```

{
  "scene_id": "scene_002",
  "source_image": "page_002.jpg",
  "mode": "crop",
  "crop": {
    "x": 100,
    "y": 250,
    "width": 1200,
    "height": 800
  },
  "duration_sec": 4.0,
  "motion": {
    "type": "pan",
    "direction": "left_to_right"
  },
  "validation": {
    "status": "approved",
    "confidence": 0.95
  }
}

```

For an uncertain crop:

</> JSON 

```

{
  "scene_id": "scene_003",
  "source_image": "page_003.jpg",
  "mode": "contain",
  "crop": null,
  "duration_sec": 6.0,
  "validation": {

```

```
"status": "needs_review",  
"confidence": 0.42  
}  
}
```

Key idea: mode must be explicit. Do not let the renderer silently decide to crop every image.

8. Better Motion Effects for Tall Panels

Even when you preserve the full image, you may want to make a tall comic panel more engaging.

Instead of splitting it randomly into multiple crops, you can use a **controlled camera movement**.

Option 1: Full-image display

Show the entire image, scaled to fit, with a background.

Best for:

- Important character reveals
- Full-body artwork
- Tall single-panel scenes
- Establishing shots

Option 2: Controlled vertical pan

Show the tall image in a viewport, then move the viewport slowly from top to bottom.

Best for:

- Long scene illustrations
- Tall artwork where details need to be readable
- Images that are too tall to display large in one frame

However, the pan should be an intentional presentation choice, not an assumption that the source consists of several independent panels. If a face or important detail is the focus, use a safe region-of-interest crop.

Option 3: Manual scene regions

If a single long image contains multiple meaningful areas, define those regions explicitly.

For example:

```
</> JSON
{
  "source_image": "long_scene.jpg",
  "regions": [
    {
      "id": "region_1",
      "y": 0,
      "height": 1100
    },
    {
      "id": "region_2",
      "y": 1100,
      "height": 900
    }
  ]
}
```

These coordinates are only examples. The regions must be derived from actual image boundaries or verified visual areas. Do not split at fixed intervals such as every 1000 pixels.

9. My Recommended Implementation Strategy

You don't need to solve all cropping problems with AI from the beginning.

Build your rendering engine in stages:

1 Safe Contain Renderer

All images preserve their complete content. Add blurred backgrounds for unused space.

Must have

2 Verified Crop Support

Support manually defined crop regions with safe boundaries and validation.

Must have

3 Controlled Pan / Zoom

Animate a viewport without assuming that every tall image should be split.

Next

4 Automatic Panel Segmentation

Detect candidate panel boundaries and validate them before splitting.

Advanced

5 Content-Aware Crop Optimization

Use subject detection and image analysis to propose better framing.

Advanced

10. Let's Fix Your Actual Cropping Problem

I want to help you build the correct solution rather than give you another generic video editor.

Please share one example of the input image that was cropped incorrectly. Upload the original Manhwa image/page and, if possible, the resulting cropped frame or video screenshot.

With a real example, I can determine whether the issue is:

- Full-page cropping
- Panel segmentation
- Incorrect aspect-ratio handling
- Motion effect framing
- Incorrect image resizing
- A combination of these problems

Then I can design the appropriate Python cropping algorithm for your case, including the safe fallback and panel detection rules.

One question: Which type of input do you usually have?

Choose your typical image format

- ☐ A very long vertical Manhwa page containing multiple scenes
- ☐ Individual Manhwa panels, one image per scene
- ☐ Both: some pages are long, others are individual panels
- ☐ Full-screen screenshots from an existing video



My typical Manhwa image format is: mixed. Help me design the correct safe panel cropping and video framing algorithm for this format.

Safe Panel Cropping & Video Framing Algorithm for Mixed Manhwa Images

Because your source images are **mixed**—individual panels, long webtoon pages, full-page screenshots, and images containing multiple panels—you should not use one cropping rule for every image.

The safest design is a **three-stage pipeline**:



1. Analyze the source image

Detect image type, panel candidates, text, faces, and important subjects.

2. Validate the crop



Check whether a proposed crop cuts off faces, characters, speech bubbles, or important objects.

3. Frame the video

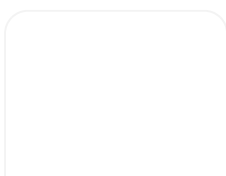


Choose contain, controlled crop, or vertical pan/zoom based on the validated result.

Core safety rule: If the algorithm is not confident that a crop is correct, preserve the complete original image instead of cropping it.

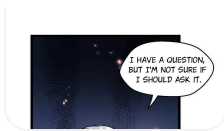
1. Use four image-processing modes

Your algorithm should classify each input into one of these modes.

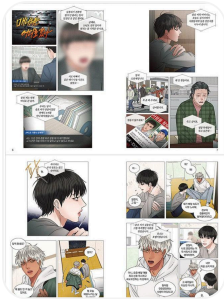


Mode A — Individual panel Safe crop

Example: One character, one action, or one dialogue scene already isolated.



- Preserve the entire image.
- Use aspect-ratio fitting.
- Apply only a small validated zoom.
- Do not detect and cut another panel from it.



Mode B — Long webtoon page Candidate segmentation

Example: A tall page containing multiple panels stacked vertically.

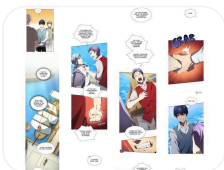
- Detect possible panel boundaries.
- Generate candidate segments.
- Validate each segment.
- Keep the entire page if boundaries are uncertain.
- Use controlled vertical movement rather than arbitrary slicing.



Mode C — Multi-panel screenshot Selective crop

Example: A landscape or rectangular screenshot containing two or more panels.

- Detect whitespace and panel borders.
- Avoid splitting through speech bubbles or characters.
- Crop only when a complete panel can be identified.
- Otherwise render the complete screenshot.



Mode D — Unknown or difficult image No automatic crop

Example: A cropped screenshot, collage, unusual layout, or image with unclear boundaries.

- Keep the complete image.
- Use a background fill.
- Add a slow zoom or pan only within safe limits.
- Mark the image for optional manual review.

2. The decision algorithm

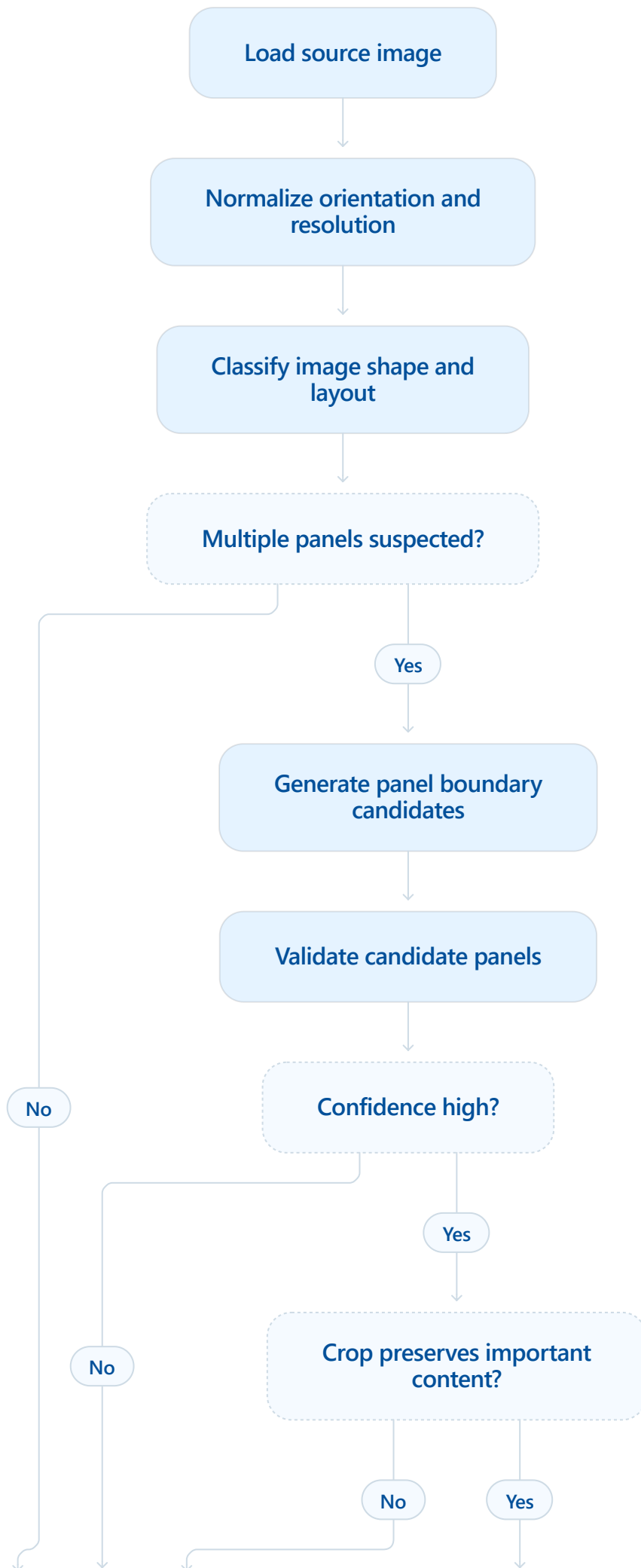
Do **not** start by asking, "How can I force this image into 16:9?"

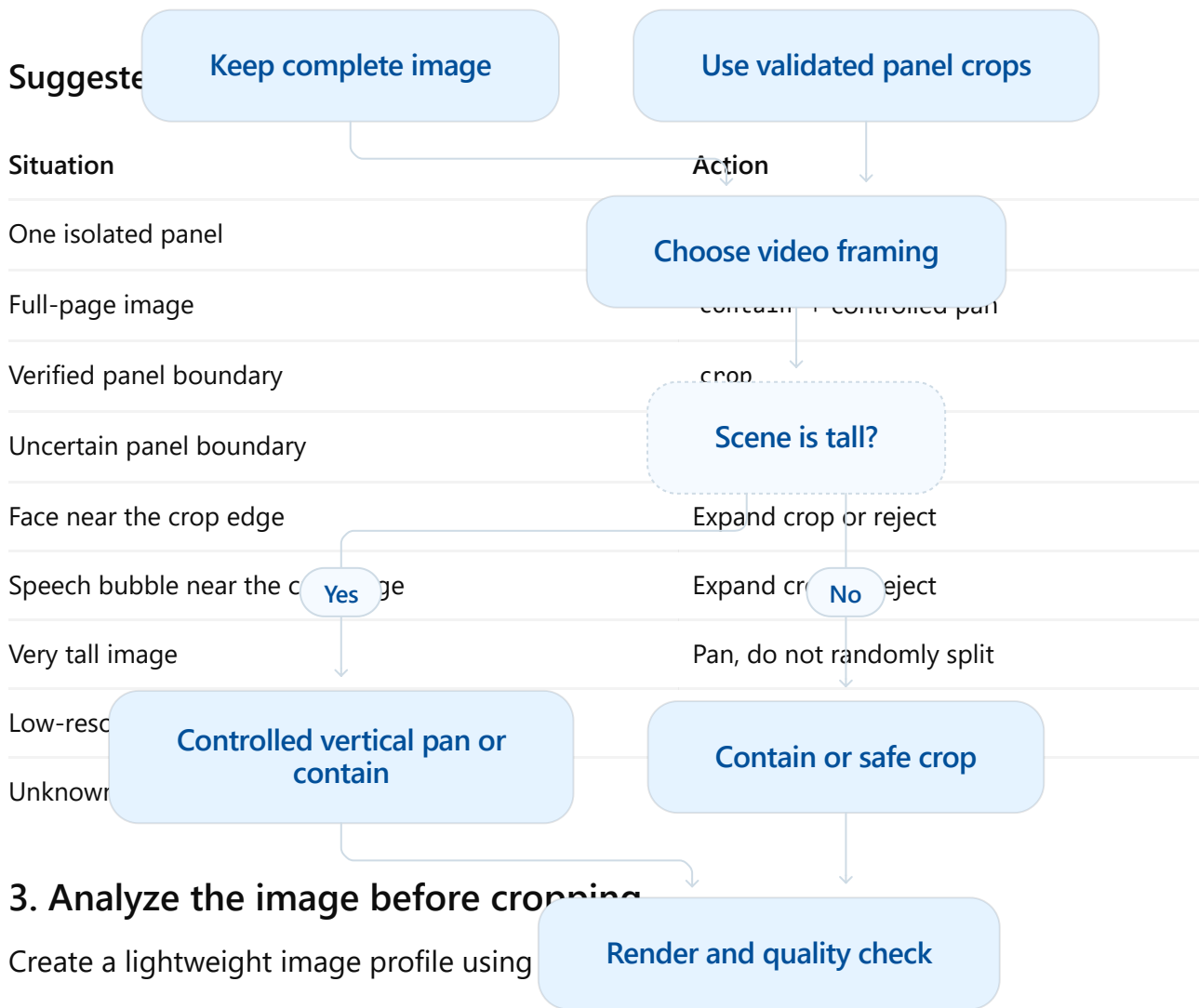
Instead, ask:

1. Is the complete source image already a single scene?
2. Are there reliable boundaries separating multiple panels?

3. Will the proposed crop preserve important visual content?
4. Is the remaining image large enough and readable?
5. Which framing method fits the video output?

Recommended decision flow





3. Analyze the image before cropping

Create a lightweight image profile using

<> Python



Run

```
from dataclasses import dataclass
from pathlib import Path
from PIL import Image

@dataclass
class ImageProfile:
    path: str
    width: int
    height: int
    aspect_ratio: float
    orientation: str
    area: int

def analyze_image(path: str) -> ImageProfile:
    with Image.open(path) as image:
        width, height = image.size

    ratio = width / height

    if ratio >= 1.25:
```

```

        orientation = "landscape"
    elif ratio <= 0.80:
        orientation = "portrait"
    else:
        orientation = "square_or_standard"

    return ImageProfile(
        path=str(path),
        width=width,
        height=height,
        aspect_ratio=ratio,
        orientation=orientation,
        area=width * height,
    )

```

Important: Image orientation alone must not determine whether an image is split. A tall image might be a single full scene, while a landscape image might contain several panels.

Use dimensions only as an initial signal, not as proof.

4. Generate panel candidates without immediately accepting them

For a mixed-format source, use multiple signals:

- Horizontal and vertical whitespace.
- Strong border lines.
- Large low-detail gaps between panels.
- Connected regions and contours.
- Speech-bubble and text positions.
- Character and face locations.
- Candidate panel size and aspect ratio.

Candidate boundary example

For a tall page, first look for horizontal gaps. However, do not simply divide the page every 20% or 33%. A white gap might be part of the artwork, and a dark line might be a character's weapon or clothing.

A simple initial candidate detector can use horizontal brightness profiles:

</> Python



▶ Run

```

import cv2
import numpy as np

```

```

def find_horizontal_gaps(
    image_path: str,
    min_gap_height: int = 12,
    brightness_threshold: int = 242,
):
    image = cv2.imread(image_path)

    if image is None:
        raise FileNotFoundError(image_path)

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Percentage of near-white pixels in each row.
    white_ratio = np.mean(gray >= brightness_threshold, axis=1)

    candidate_ranges = []
    start = None

    for y, ratio in enumerate(white_ratio):
        is_gap = ratio >= 0.92

        if is_gap and start is None:
            start = y

        elif not is_gap and start is not None:
            end = y

            if end - start >= min_gap_height:
                candidate_ranges.append((start, end))

            start = None

    if start is not None:
        end = len(white_ratio)

        if end - start >= min_gap_height:
            candidate_ranges.append((start, end))

    return candidate_ranges

```

This is **only a candidate generator**. It is not safe enough to crop directly because:

- Many manhwa panels use colored backgrounds.
- White areas can appear inside a panel.
- A gap may be irregular or very small.
- Some pages have no visible gutters.
- Text and characters can cross expected boundaries.

The next step must validate each candidate.

5. Use a safety margin around every crop

Never crop exactly to the detected panel boundary. Expand it slightly to protect nearby content.

</> Python



▶ Run

```
def expand_box(box, image_width, image_height, margin_ratio=0.04):
    x1, y1, x2, y2 = box

    margin_x = int((x2 - x1) * margin_ratio)
    margin_y = int((y2 - y1) * margin_ratio)

    x1 = max(0, x1 - margin_x)
    y1 = max(0, y1 - margin_y)
    x2 = min(image_width, x2 + margin_x)
    y2 = min(image_height, y2 + margin_y)

    return x1, y1, x2, y2
```

For your first version, use a margin of approximately 4–8%, then adjust based on results.

The margin should be constrained by neighboring panels. Expanding too far could accidentally include content from the next panel, so the algorithm should also check for overlap with adjacent candidates.

6. Validate crops before accepting them

This is the most important part of your system.

A crop should be rejected if it is likely to cut off important information.

Validation rules

Check	Recommended behavior
Crop width or height too small	Reject
Very low resolution	Reject
Crop removes large parts of a face	Reject
Speech bubble touches crop edge	Expand or reject
Character is cut at an unusual location	Expand or reject

Crop contains only a narrow strip	Reject
Crop overlaps another panel heavily	Reconsider
Candidate is nearly identical to full image	Keep full image
Confidence is below threshold	Use contain

Safe crop model

Represent every crop using coordinates relative to the original image:

 Python



 Run

```
from dataclasses import dataclass
from typing import Optional

@dataclass
class CropCandidate:
    x1: int
    y1: int
    x2: int
    y2: int
    confidence: float
    reason: str
    approved: bool = False
```

A crop should not be approved only because its confidence score is high. Use multiple checks.

 Python



 Run

```
def basic_crop_validation(
    box,
    image_width,
    image_height,
    min_width=240,
    min_height=240,
    min_area_ratio=0.08,
):
    x1, y1, x2, y2 = box

    width = x2 - x1
    height = y2 - y1
    area = width * height

    if width < min_width or height < min_height:
        return False, "Crop resolution is too small"
```

```

if x1 < 0 or y1 < 0:
    return False, "Crop starts outside image"

if x2 > image_width or y2 > image_height:
    return False, "Crop ends outside image"

if x2 <= x1 or y2 <= y1:
    return False, "Invalid crop coordinates"

if area / (image_width * image_height) < min_area_ratio:
    return False, "Crop is too small relative to source"

return True, "Basic checks passed"

```

This basic validator does not detect faces or speech bubbles yet. Add those detectors in a later stage, and treat their results as additional evidence rather than absolute truth.

7. Use a crop confidence score

Instead of using a simple yes/no decision from one detector, calculate a confidence score from several signals.

For example:

Signal	Weight
Clear gutter or panel boundary	25%
Crop contains a complete region	20%
No important content near edges	25%
Adequate resolution	15%
Low overlap with neighboring panels	15%

These weights are initial engineering values, not universal truths. Tune them using your own manhwa samples.

</> Python



▶ Run

```

def calculate_crop_confidence(
    boundary_score,
    content_safety_score,
    edge_safety_score,
    resolution_score,
    overlap_score,

```

```

):
    weights = {
        "boundary": 0.25,
        "content": 0.20,
        "edge": 0.25,
        "resolution": 0.15,
        "overlap": 0.15,
    }

    confidence = (
        boundary_score * weights["boundary"]
        + content_safety_score * weights["content"]
        + edge_safety_score * weights["edge"]
        + resolution_score * weights["resolution"]
        + overlap_score * weights["overlap"]
    )

    return round(confidence, 3)

```

Use the result conservatively:

</> Python



▶ Run

```

def choose_crop_action(confidence):
    if confidence >= 0.85:
        return "approved_crop"

    if confidence >= 0.65:
        return "manual_review"

    return "full_image_contain"

```

For a fully automatic YouTube pipeline, I recommend initially using:

- **0.85 or higher:** Crop if all hard safety checks pass.
- **0.65–0.849:** Preserve the full image or send it for review.
- **Below 0.65:** Do not crop.

A confidence score should never override a hard safety failure. For example, if a crop cuts off a face, reject it regardless of the score.

8. Build a safe video framing system

Your video renderer should support three primary framing modes.

A. Contain

Default

Show the complete image without cutting any pixels. Fill unused areas with a blurred version of the same image or a solid background.

B. Validated crop

Conditional

Use only a verified crop box with safety margins. This is appropriate for clearly separated panels.

C. Controlled pan and zoom

Tall images

Keep the entire source image and move the viewport slowly over it. Never generate random crops for every frame.

9. Aspect-ratio-preserving contain renderer

For a 16:9 video, you can render any source image without cutting it.

The following Pillow implementation:

- Preserves the original aspect ratio.
- Creates a blurred background.
- Places the complete source image in the center.
- Avoids forced cropping.

</> Python



▶ Run

```
from PIL import Image, ImageFilter

def create_contain_frame(
    image_path: str,
    output_size=(1920, 1080),
    background_blur=24,
):
    target_width, target_height = output_size

    source = Image.open(image_path).convert("RGB")

    # Create a blurred background from the same image.
    background = source.copy()
    background.thumbnail(output_size, Image.Resampling.LANCZOS)

    # Scale background to cover the output.
    bg_ratio = max(
        target_width / background.width,
        target_height / background.height,
    )
```

```

bg_size = (
    max(1, int(background.width * bg_ratio)),
    max(1, int(background.height * bg_ratio)),
)

background = background.resize(bg_size, Image.Resampling.LANCZOS)

left = (background.width - target_width) // 2
top = (background.height - target_height) // 2

background = background.crop(
    (left, top, left + target_width, top + target_height)
)

background = background.filter(
    ImageFilter.GaussianBlur(background_blur)
)

# Fit the original image completely inside the output.
foreground = source.copy()
foreground.thumbnail(output_size, Image.Resampling.LANCZOS)

x = (target_width - foreground.width) // 2
y = (target_height - foreground.height) // 2

background.paste(foreground, (x, y))

return background

```

This should be your initial default for **every uncertain image**.

Why contain matters

Your earlier problem was that a long image or a character was being split into several arbitrary images. A contain renderer ensures that:

- A face is not automatically cut in half.
- Buildings and backgrounds remain visible.
- Full-page images can be displayed without destroying composition.
- The renderer does not assume that all source images have the same aspect ratio.

10. Implement controlled vertical movement for tall pages

A long page does not always need to be divided into separate panels. Sometimes it is better to show the full page using a slow vertical camera movement.

The idea is:

1. Scale the source to fit the video width.
2. Calculate the visible viewport.
3. Move from the top toward the bottom.
4. Keep the movement slow.
5. Avoid showing a crop that cuts through important content abruptly.

Camera path structure

</> Python



▶ Run

```
from dataclasses import dataclass

@dataclass
class CameraPath:
    start_y: float
    end_y: float
    start_scale: float = 1.0
    end_scale: float = 1.04
    duration_sec: float = 5.0
```

For an initial implementation, use:

- Zoom range: approximately 1.00 to 1.05 .
- Slow movement rather than rapid movement.
- A minimum scene duration of around 4–6 seconds for complex artwork.
- No more than one major vertical movement per scene unless the image is very tall.

Avoid zooming into a face automatically until you have a reliable subject detector. An incorrect focal point can be more distracting than a full-image view.

11. Use a scene JSON format

Do not store only a list of image filenames. Save the rendering decision alongside each image.

</> JSON



```
{
  "scene_id": "scene_001",
  "source_image": "page_001.jpg",
  "mode": "contain",
  "crop": null,
```

```

"duration_sec": 5.5,
"motion": {
  "type": "slow_zoom",
  "start_scale": 1.0,
  "end_scale": 1.04,
  "start_y": 0.5,
  "end_y": 0.5
},
"validation": {
  "status": "approved",
  "confidence": 1.0,
  "reason": "Full image preserved"
}
}

```

For a validated crop:

</> JSON 

```

{
  "scene_id": "scene_002",
  "source_image": "page_002.jpg",
  "mode": "crop",
  "crop": {
    "x1": 120,
    "y1": 640,
    "x2": 980,
    "y2": 1420
  },
  "duration_sec": 4.8,
  "motion": {
    "type": "slow_zoom",
    "start_scale": 1.0,
    "end_scale": 1.03
  },
  "validation": {
    "status": "approved",
    "confidence": 0.91,
    "reason": "Verified panel boundary and edge safety"
  }
}

```

For an uncertain candidate:

</> JSON 

```

{
  "scene_id": "scene_003",
  "source_image": "page_003.jpg",
  "mode": "contain",

```

```
"crop": null,  
"duration_sec": 5.0,  
"motion": {  
  "type": "none"  
},  
"validation": {  
  "status": "needs_review",  
  "confidence": 0.58,  
  "reason": "Panel boundaries are ambiguous"  
}  
}
```

This is useful because you can later inspect exactly **why** an image was cropped or preserved.

12. Recommended complete pipeline

1 Load and normalize

Read the image, correct orientation, remove unsupported color formats, and record its original dimensions.

2 Classify the layout

Identify whether the image is likely an individual panel, multi-panel page, long page, or unknown layout.

3 Generate candidates

Search for panel gutters using whitespace, contours, borders, and layout analysis.

4 Create expanded candidate boxes

Add safety margins while preventing excessive overlap with neighboring panels.

5 Run hard validation

Reject low-resolution, invalid, extremely narrow, or content-cutting crops.

6 Score remaining candidates

Calculate boundary, content, edge, resolution, and overlap scores.

7 Select the rendering mode

Use a validated crop only when confidence and hard checks pass. Otherwise choose contain or controlled pan.

8 Create the scene JSON

Store the image, rendering mode, crop coordinates, duration, camera path, and validation details.

9 Render the video

Generate the scene with FFmpeg, MoviePy, or OpenCV.

10 Run an output quality check

Inspect the first, middle, and final frames of each scene. Flag scenes with blank output, excessive zoom, or clipped content.

13. Build it in stages

I recommend not implementing the complete AI panel detection system immediately.

Phase 1 Safe rendering first

Implement `contain`, blurred backgrounds, and basic zoom. Disable automatic cropping.

Phase 2 Manual crop JSON

Allow verified crop coordinates to be entered manually. Test whether the video renderer respects them.

Phase 3 Candidate detection

Detect gutters and panel boundaries, but preserve the full image by default.

Phase 4 Safety validation

Add text, face, subject, and edge-protection checks. Compare crops against the original image.

Phase 5 Automatic panel segmentation

Enable automatic cropping only for formats where validation performs reliably.

14. Suggested project structure

```
manhwa_video/  
├──  
├── input/  
│   ├── page_001.jpg  
│   └── page_002.png  
└──
```



```
|— analysis/
|   |— image_profile.py
|   |— panel_candidates.py
|   |— crop_validator.py
|
|— framing/
|   |— contain.py
|   |— crop.py
|   |— pan_zoom.py
|   |— camera_path.py
|
|— scenes/
|   |— scenes.json
|
|— output/
|   |— previews/
|   |— final_video.mp4
|
|— main.py
```

My recommended default configuration

Use this as the first safe configuration for your mixed source format:

</> Python



▶ Run

```
CONFIG = {
    "output_size": (1920, 1080),
    "default_mode": "contain",
    "allow_automatic_crop": False,
    "minimum_crop_confidence": 0.85,
    "crop_margin_ratio": 0.06,
    "minimum_crop_width": 240,
    "minimum_crop_height": 240,
    "use_blurred_background": True,
    "allow_vertical_pan": True,
    "maximum_zoom": 1.05,
    "preserve_full_image_on_failure": True,
    "save_debug_previews": True,
}
```

The most important setting is `preserve_full_image_on_failure: True`. Your video may occasionally contain an image that is not perfectly framed, but it should not destroy faces, buildings, speech bubbles, or other story details through an incorrect crop.

Final recommendation

Start with this priority:

Complete image preservation → validated cropping → controlled pan/zoom → automatic segmentation.

Do not begin with automatic cropping as the default. For mixed manhwa images, a safe renderer with occasional full-page scenes is more reliable than a system that aggressively splits every source image and produces visually damaged frames.



ChatGPT can make mistakes. Check important info.